Prompting best practices

Comprehensive guide to prompt engineering techniques for Claude's latest models, covering clarity, examples, XML structuring, thinking, and agentic systems.

Copy page

This is the single reference for prompt engineering with Claude's latest models, including Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 4.6, and Claude Haiku 4.5. It covers foundational techniques, output control, tool use, thinking, and agentic systems. Jump to the section that matches your situation.

For an overview of model capabilities, see the models overview. For details on what's new in Claude Opus 4.8, see What's new in Claude Opus 4.8. For migration guidance, see the Migration guide.

Prompting Claude Opus 4.8

Claude Opus 4.8 has particular strengths in long-horizon agentic work, knowledge work, vision, and memory tasks. It performs well out of the box on existing Claude Opus 4.7 prompts. The patterns below cover the behaviors that most often require tuning.

For API parameter changes when migrating from Claude Opus 4.7 (sampling parameters, effort default, 1M context window default (200k on Microsoft Foundry), mid-conversation system messages, and refusal stop details), see the migration guide.

Response length and verbosity

Claude Opus 4.8 calibrates response length to how complex it judges the task to be, rather than defaulting to a fixed verbosity. This usually means shorter answers on simple lookups and much longer ones on open-ended analysis.

If your product depends on a certain style or verbosity of output, you may need to
As an example, to decrease verbosity, you might add:

```
Provide concise, focused responses. Skip non-essential context, and keep mp
```

If you see specific examples of kinds of verbosity (i.e. over-explaining), you can add additional instructions in your prompt to prevent them. Positive examples showing how Claude can communicate with the appropriate level of concision tend to be more effective than negative examples or instructions that tell the model what not to do.

Calibrating effort and thinking depth

The effort parameter allows you to tune Claude's intelligence vs. token spend, trading off capability for faster speed and lower costs. Start with the `xhigh` effort level for coding and agentic use cases, and use a minimum of `high` effort for most intelligence-sensitive use cases. Experiment with other effort levels to further tune token usage and intelligence:

- `max`: Max effort can deliver performance gains in some use cases, but may show diminishing returns from increased token usage. This setting can also sometimes be prone to overthinking. Test max effort for intelligence-demanding tasks.
- `xhigh`: Extra high effort is the best setting for most coding and agentic use cases.
- `high`: This setting balances token usage and intelligence. For most intelligence-sensitive use cases, use a minimum of `high` effort.
- `medium`: Good for cost-sensitive use cases that need to reduce token usage while trading off intelligence.
- `low`: Reserve for short, scoped tasks and latency-sensitive workloads that are not intelligence-sensitive.

Claude Opus 4.8 respects effort levels strictly, especially at the low end. At `low` and `medium`, the model scopes its work to what was asked rather than going above and beyond. This is good for latency and cost, but on moderately complex tasks running at `low` effort there is some risk of under-thinking.

If you observe shallow reasoning on complex problems, raise effort to `high` or `xhigh` rather than prompting around it. If you need to keep effort at `low` for latency, add targeted guidance:

```
This task involves multi-step reasoning. Think carefully through the prob b
```

Effort is likely to be more important for this model than for any prior Opus, so experiment with it actively when you upgrade.

On Claude Opus 4.8, thinking is off unless you explicitly set `thinking: {type: "adaptive"}`. The triggering behavior for adaptive thinking is steerable. If you find the model thinking more often than you'd like, which can happen with large or complex system prompts, add guidance to steer it. As always, measure the effect of any prompting changes on performance. Example:

```
Thinking adds latency and should only be used when it will meaningfully i ov
```

Conversely, if you're running hard workloads at `medium` and seeing under-thinking, the first lever is to raise effort. If you need finer control, prompt for it directly.

> If you are running Claude Opus 4.8 at `max` or `xhigh` effort, set a large max output token budget so the model has room to think and act across its subagents and tool calls. Start at 64k tokens and tune from there.

Tool use triggering

Claude Opus 4.8 has a tendency to favor reasoning over tool calls. This produces better results in most cases. However, increasing the effort setting is a useful lever to increase the level of tool usage, especially in knowledge work. `high` or `xhigh` effort settings show substantially more tool usage in agentic search and coding. For scenarios where you want more tool use, you can also adjust your prompt to explicitly instruct the model about when and how to properly use its tools. For instance, if you find that the model is not using your web search tools, clearly describe why and how it should.

User-facing progress updates

Claude Opus 4.8 provides more regular, higher-quality updates to the user throughout long agentic traces. If you've added scaffolding to force interim status messages ("After every 3 tool calls, summarize progress"), try removing it. If you find that the length or contents of Claude Opus 4.8's user-facing updates are not well-calibrated to your use case, explicitly describe what these updates should look like in the prompt and provide examples.

More literal instruction following

Claude Opus 4.8 interprets prompts literally and explicitly, particularly at lower effort levels. It does not silently generalize an instruction from one item to another, and it does not infer requests you didn't make. The upside of this literalism is precision and less thrash, and it generally performs better for API use cases with carefully tuned prompts, structured extraction, and pipelines where you want predictable behavior. If you need Claude to apply an instruction broadly, state the scope explicitly (for example, "Apply this formatting to every section, not just the first one").

Tone and writing style

As with any new model, prose style on long-form writing may shift. Claude Opus 4.8 tends toward a direct, opinionated style with minimal validation-forward phrasing and sparing emoji use. If your product relies on a specific voice, re-evaluate style prompts against the new baseline.

For instance, if your product voice is warmer or more conversational, add:

```
Use a warm, collaborative tone. Acknowledge the user's framing before ans in
```

Controlling subagent spawning

Claude Opus 4.8 tends to spawn fewer subagents by default. However, this behavior is steerable through prompting; give Claude Opus 4.8 explicit guidance around when subagents are desirable. A toy example for a coding use case:

```
Do not spawn a subagent for work you can complete directly in a single re ns

Spawn multiple subagents in the same turn when fanning out across items or rea
```

Design and frontend defaults

Claude Opus 4.8 has strong design instincts, with a consistent default house style: warm cream/off-white backgrounds (~ `#F4F1EA`), serif display type (Georgia, Fraunces, Playfair), italic word-accents, and a terracotta/amber accent. This reads well for editorial, hospitality, and portfolio briefs, but will feel off for dashboards, dev tools, fintech, healthcare, or enterprise apps. The default appears in slide decks as well as web UIs.

This default is persistent. Generic instructions ("don't use cream," "make it clean and minimal") tend to shift the model to a different fixed palette rather than producing variety. Two approaches work reliably:

1. Specify a concrete alternative. The model follows explicit specs precisely:

```
Design a desktop landing page for a supplement brand called AEFRM.

The visual direction should come from a cold monochrome atmosphere using pale

The page should feel sharp and controlled, with a strong sense of structure an

Use this tonal system across the full page instead of introducing bright accen

Use the uploaded image on the hero design in black and white.

The layout should be built with clear horizontal sections and a centered max-w

Typography should use a square, angular sans-serif with wider letter spacing t

For the structure, start with a hero section containing a strong product state

Buttons should be flat and precise, with subtle hover changes using transition
```

```
Color palette should stay within this range:
#E9ECEC, #C9D2D4, #8C9A9E, #44545B, #11171B.
```

2. Have the model propose options before building. This breaks the default and gives users control. If you previously relied on `temperature` for design variety, use this approach; it produces meaningfully different directions across runs. Example prompt:

```
Before building, propose 4 distinct visual directions tailored to this br (
```

Additionally, Claude Opus 4.8 requires less frontend design prompting than previous models to avoid generic patterns that users call the "AI slop" aesthetic. With earlier models, Anthropic recommended a lengthier prompt snippet in the frontend-design skill. However, Claude Opus 4.8 generates distinctive, creative frontends with more minimal prompting guidance. This prompt snippet works well with the above prompting advice for variety:

```
<frontend_aesthetics>
NEVER use generic AI-generated aesthetics like overused font families (Inter,
</frontend_aesthetics>
```

Interactive coding products

Claude Opus 4.8's token usage and behavior can differ between autonomous, asynchronous coding agents with a single user turn and interactive, synchronous coding agents with multiple user turns. Specifically, it tends to use more tokens in interactive settings, primarily because it reasons more after user turns. This can improve long-horizon coherence, instruction following, and coding capabilities in long, interactive coding sessions, but also comes with more token usage. To maximize both performance and token efficiency in coding products, use `xhigh` or `high` effort, add autonomous features like an auto mode, and reduce the number of human interactions required from your users.

Of course, when limiting the number of required user interactions, it's important to specify the task, intent, and relevant constraints upfront in the first human turn. Providing well-specified, clear, and accurate task descriptions upfront can help maximize autonomy and intelligence while minimizing extra token usage after user turns. Because Claude Opus 4.8 is more autonomous than prior models, this usage pattern helps to maximize performance. In contrast, ambiguous or underspecified prompts conveyed progressively over multiple user turns tend to relatively reduce token efficiency and sometimes performance.

Code review harnesses

Claude Opus 4.8 is meaningfully better at finding bugs than prior models, and has both higher recall

and precision in internal evals. However, if your code-review harness was tuned for an earlier model, you may initially see lower recall. This is likely a harness effect, not a capability regression. When a review prompt says things like "only report high-severity issues," "be conservative," or "don't nitpick," Claude Opus 4.8 may follow that instruction more faithfully than earlier models did: it may investigate the code just as thoroughly, identify the bugs, and then not report findings it judges to be below your stated bar. This can show up as the model doing the same depth of investigation but converting fewer investigations into reported findings, especially on lower-severity bugs. Precision typically rises, but measured recall can fall even though the model's underlying bug-finding ability has improved.

Some recommended prompt language:

```
Report every issue you find, including ones you are uncertain about or co de
```

This prompt can be used without having an actual second step, but moving confidence filtering out of the finding step often helps. If your harness has a separate verification, deduplication, or ranking stage, tell the model explicitly that its job at the finding stage is coverage rather than filtering.

If you do want the model to self-filter in a single pass, be concrete about where the bar is rather than using qualitative terms like "important": for example, "report any bugs that could cause incorrect behavior, a test failure, or a misleading result; only omit nits like pure style or naming preferences."

Iterate on prompts against a subset of your evals or test cases to validate recall or F1 score gains.

Computer use

Computer use capability works across resolutions, up to a maximum resolution of 2576px / 3.75MP. Internal computer use testing shows that sending images at 1080p provides a good balance of performance and cost.

For particularly cost-sensitive workloads, 720p or 1366×768 are lower-cost options with strong performance. Conduct your own testing to find the ideal settings for your use case; experimenting with effort settings can also help tune the model's behavior.

General principles

Be clear and direct

Claude responds well to clear, explicit instructions. Being specific about your desired output can help enhance results. If you want "above and beyond" behavior, explicitly request it rather than relying on the model to infer this from vague prompts.

Think of Claude as a brilliant but new employee who lacks context on your norms and workflows. The more precisely you explain what you want, the better the result.

Golden rule: Show your prompt to a colleague with minimal context on the task and ask them to follow it. If they'd be confused, Claude will be too.

- Be specific about the desired output format and constraints.

- Provide instructions as sequential steps using numbered lists or bullet points when the order or completeness of steps matters.

> **Example: Creating an analytics dashboard**

Add context to improve performance

Providing context or motivation behind your instructions, such as explaining to Claude why such behavior is important, can help Claude better understand your goals and deliver more targeted responses.

> **Example: Formatting preferences**

Claude is smart enough to generalize from the explanation.

Use examples effectively

Examples are one of the most reliable ways to steer Claude's output format, tone, and structure. A few well-crafted examples (known as few-shot or multishot prompting) can dramatically improve accuracy and consistency.

When adding examples, make them:

- **Relevant:** Mirror your actual use case closely.
- **Diverse:** Cover edge cases and vary enough that Claude doesn't pick up unintended patterns.
- **Structured:** Wrap examples in `<example>` tags (multiple examples in `<examples>` tags) so Claude can distinguish them from instructions.

> Include 3–5 examples for best results. You can also ask Claude to evaluate your examples for relevance and diversity, or to generate additional ones based on your initial set.

Structure prompts with XML tags

XML tags help Claude parse complex prompts unambiguously, especially when your prompt mixes instructions, context, examples, and variable inputs. Wrapping each type of content in its own tag (e.g. `<instructions>`, `<context>`, `<input>`) reduces misinterpretation.

Best practices:

- Use consistent, descriptive tag names across your prompts.
- Nest tags when content has a natural hierarchy (documents inside `<documents>`, each inside `<document index="n">`).

Give Claude a role

Setting a role in the system prompt focuses Claude's behavior and tone for your use case. Even a single sentence makes a difference:

Python

```python
import anthropic

client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=1024,
    system="You are a helpful coding assistant specializing in Python.",
    messages=[
        {"role": "user", "content": "How do I sort a list of dictionaries by k
    ],
)
print(message.content)
```

Long context prompting

When working with large documents or data-rich inputs (20k+ tokens), structure your prompt carefully to get the best results:

- **Put longform data at the top:** Place your long documents and inputs near the top of your prompt, above your query, instructions, and examples. This can significantly improve performance across all models.

  > ⓘ Queries at the end can improve response quality by up to 30% in tests, especially with complex, multi-document inputs.

- **Structure document content and metadata with XML tags:** When using multiple documents, wrap each document in `<document>` tags with `<document_content>` and `<source>` (and other metadata) subtags for clarity.

  > **Example multi-document structure**

- **Ground responses in quotes:** For long document tasks, ask Claude to quote relevant parts of the documents first before carrying out its task. This helps Claude cut through the noise of the rest of the document's contents.

> **Example quote extraction**

Model self-knowledge

If you would like Claude to identify itself correctly in your application or use specific API strings:

Sample prompt for model identity

```
The assistant is Claude, created by Anthropic. The current model is Claude Opu
```

For LLM-powered apps that need to specify model strings:

Sample prompt for model string

```
When an LLM is needed, please default to Claude Opus 4.8 unless the user reque
```

Output and formatting

Communication style and verbosity

Claude's latest models have a more concise and natural communication style compared to previous models:

- **More direct and grounded:** Provides fact-based progress reports rather than self-celebratory updates
- **More conversational:** Slightly more fluent and colloquial, less machine-like
- **Less verbose:** May skip detailed summaries for efficiency unless prompted otherwise

This means Claude may skip verbal summaries after tool calls, jumping directly to the next action. If you prefer more visibility into its reasoning:

Sample prompt

```
After completing a task that involves tool use, provide a quick summary of the
```

Control the format of responses

There are a few particularly effective ways to steer output formatting:

1. **Tell Claude what to do instead of what not to do**
    - Instead of: "Do not use markdown in your response"
    - Try: "Your response should be composed of smoothly flowing prose paragraphs."
2. **Use XML format indicators**
    - Try: "Write the prose sections of your response in <smoothly_flowing_prose_paragraphs> tags."
3. **Match your prompt style to the desired output**

    The formatting style used in your prompt may influence Claude's response style. If you are still experiencing steerability issues with output formatting, try matching your prompt style to your desired output style as closely as possible. For example, removing markdown from your prompt can reduce the volume of markdown in the output.
4. **Use detailed prompts for specific formatting preferences**

    For more control over markdown and formatting usage, provide explicit guidance:

Sample prompt to minimize markdown

```
<avoid_excessive_markdown_and_bullet_points>
When writing reports, documents, technical explanations, analyses, or any long

DO NOT use ordered lists (1. ...) or unordered lists (*) unless : a) you're pr

Instead of listing items with bullets or numbers, incorporate them naturally i

Your goal is readable, flowing text that guides the reader naturally through i
</avoid_excessive_markdown_and_bullet_points>
```

LaTeX output

Claude's latest models default to LaTeX for mathematical expressions, equations, and technical explanations. If you prefer plain text, add the following instructions to your prompt:

Sample prompt

```
Format your response in plain text only. Do not use LaTeX, MathJax, or any mar
```

Document creation

Claude's latest models excel at creating presentations, animations, and visual documents with impressive creative flair and strong instruction following. The models produce polished, usable output on the first try in most cases.

For best results with document creation:

Sample prompt

```
Create a professional presentation on [topic]. Include thoughtful design eleme
```

Migrating away from prefilled responses

Starting with Claude 4.6 models and Claude Mythos Preview, prefilled responses on the last assistant turn are no longer supported. Requests with prefilled assistant messages to these models return a 400 error. Model intelligence and instruction following have advanced such that most use cases of prefill no longer require it. Earlier models continue to support prefills, and adding assistant messages elsewhere in the conversation is not affected.

Here are common prefill scenarios and how to migrate away from them:

- **Controlling output formatting**
- **Eliminating preambles**
- **Avoiding bad refusals**
- **Continuations**
- **Context hydration and role consistency**

Tool use

Tool usage

Claude's latest models are trained for precise instruction following and benefit from explicit direction to use specific tools. If you say "can you suggest some changes," Claude will sometimes provide suggestions rather than implementing them, even if making changes might be what you intended.

For Claude to take action, be more explicit:

> **Example: Explicit instructions**

To make Claude more proactive about taking action by default, you can add this to your system prompt:

Sample prompt for proactive action

```
<default_to_action>
By default, implement changes rather than only suggesting them. If the user's
</default_to_action>
```

On the other hand, if you want the model to be more hesitant by default, less prone to jumping straight into implementations, and only take action if requested, you can steer this behavior with a prompt like the below:

Sample prompt for conservative action

```
<do_not_act_before_instructions>
Do not jump into implementation or change files unless clearly instructed to m
</do_not_act_before_instructions>
```

Claude Opus 4.5 and Claude Opus 4.6 are also more responsive to the system prompt than previous models. If your prompts were designed to reduce undertriggering on tools or skills, these models may now overtrigger. The fix is to dial back any aggressive language. Where you might have said "CRITICAL: You MUST use this tool when...", you can use more normal prompting like "Use this tool when...".

Optimize parallel tool calling

Claude's latest models excel at parallel tool execution. These models will:

- Run multiple speculative searches during research
- Read several files at once to build context faster
- Execute bash commands in parallel (which can even bottleneck system performance)

This behavior is easily steerable. While the model has a high success rate in parallel tool calling without prompting, you can boost this to ~100% or adjust the aggression level:

Sample prompt for maximum parallel efficiency

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies between the
</use_parallel_tool_calls>
```

Sample prompt to reduce parallel execution

```
Execute operations sequentially with brief pauses between each step to ensure
```

Thinking and reasoning

Overthinking and excessive thoroughness

Claude Opus 4.6 does significantly more upfront exploration than previous models, especially at higher `effort` settings. This initial work often helps to optimize the final results, but the model may gather extensive context or pursue multiple threads of research without being prompted. If your prompts previously encouraged the model to be more thorough, you should tune that guidance for Claude Opus 4.6:

- **Replace blanket defaults with more targeted instructions.** Instead of "Default to using [tool]," add guidance like "Use [tool] when it would enhance your understanding of the problem."
- **Remove over-prompting.** Tools that undertriggered in previous models are likely to trigger appropriately now. Instructions like "If in doubt, use [tool]" will cause overtriggering.
- **Use effort as a fallback.** If Claude continues to be overly aggressive, use a lower setting for `effort`.

In some cases, Claude Opus 4.6 may think extensively, which can inflate thinking tokens and slow down responses. If this behavior is undesirable, you can add explicit instructions to constrain its reasoning, or you can lower the `effort` setting to reduce overall thinking and token usage.

Sample prompt

```
When you're deciding how to approach a problem, choose an approach and commit
```

If you need a hard ceiling on thinking costs, extended thinking with a `budget_tokens` cap is still functional on Opus 4.6 and Sonnet 4.6 but is deprecated. Prefer lowering the effort setting or using `max_tokens` as a hard limit with adaptive thinking.

Leverage thinking & interleaved thinking capabilities

Claude's latest models offer thinking capabilities that can be especially helpful for tasks involving reflection after tool use or complex multi-step reasoning. You can guide its initial or interleaved thinking for better results.

Claude Opus 4.6 and Claude Sonnet 4.6 use adaptive thinking (`thinking: {type: "adaptive"}`), where Claude dynamically decides when and how much to think. Claude calibrates its thinking based on two factors: the `effort` parameter and query complexity. Higher effort elicits more thinking, and more complex queries do the same. On easier queries that don't require thinking, the model responds directly. In internal evaluations, adaptive thinking reliably drives better performance than extended thinking. Consider moving to adaptive thinking to get the most intelligent responses.

Use adaptive thinking for workloads that require agentic behavior such as multi-step tool use, complex coding tasks, and long-horizon agent loops. Older models use manual thinking mode with `budget_tokens` .

You can guide Claude's thinking behavior:

Example prompt

```
After receiving tool results, carefully reflect on their quality and determine
```

The triggering behavior for adaptive thinking is promptable. If you find the model thinking more often than you'd like, which can happen with large or complex system prompts, add guidance to steer it:

Sample prompt

```
Extended thinking adds latency and should only be used when it will meaningful
```

If you are migrating from extended thinking with `budget_tokens` , replace your thinking configuration and move budget control to `effort` :

Before (extended thinking, older models):

Python

```python
client.messages.create(
    model="claude-sonnet-4-5-20250929",
    max_tokens=64000,
    thinking={"type": "enabled", "budget_tokens": 32000},
    messages=[{"role": "user", "content": "..."}],
)
```

After (adaptive thinking):

Python

```python
client.messages.create(
    model="claude-opus-4-8",
    max_tokens=64000,
    thinking={"type": "adaptive"},
    output_config={"effort": "high"},  # or "max", "xhigh", "medium", "low"
    messages=[{"role": "user", "content": "..."}],
)
```

If you are not using extended thinking, no changes are required. Thinking is off by default when you omit the `thinking` parameter.

- **Prefer general instructions over prescriptive steps.** A prompt like "think thoroughly" often produces better reasoning than a hand-written step-by-step plan. Claude's reasoning frequently exceeds what a human would prescribe.
- **Multishot examples work with thinking.** Use `<thinking>` tags inside your few-shot examples to show Claude the reasoning pattern. It will generalize that style to its own extended thinking blocks.
- **Manual CoT as a fallback.** When thinking is off, you can still encourage step-by-step reasoning by asking Claude to think through the problem. Use structured tags like `<thinking>` and `<answer>` to cleanly separate reasoning from the final output.
- **Ask Claude to self-check.** Append something like "Before you finish, verify your answer against [test criteria]." This catches errors reliably, especially for coding and math.

> When extended thinking is disabled, Claude Opus 4.5 is particularly sensitive to the word "think" and its variants. Consider using alternatives like "consider," "evaluate," or "reason through" in those cases.

> For more information on thinking capabilities, see Extended thinking and Adaptive thinking.

Agentic systems

Long-horizon reasoning and state tracking

Claude's latest models excel at long-horizon reasoning tasks with exceptional state tracking capabilities. Claude maintains orientation across extended sessions by focusing on incremental progress, making steady advances on a few things at a time rather than attempting everything at once. This capability especially emerges over multiple context windows or task iterations, where Claude can work on a complex task, save the state, and continue with a fresh context window.

Context awareness and multi-window workflows

Claude 4.6 and Claude 4.5 models feature context awareness, enabling the model to track its remaining context window (i.e. "token budget") throughout a conversation. This enables Claude to execute tasks and manage context more effectively by understanding how much space it has to work.

Managing context limits:

If you are using Claude in an agent harness that compacts context or allows saving context to external files (like in Claude Code), consider adding this information to your prompt so Claude can behave accordingly. Otherwise, Claude may sometimes naturally try to wrap up work as it approaches the context limit. Below is an example prompt:

Sample prompt

```
Your context window will be automatically compacted as it approaches its limit
```

The memory tool pairs naturally with context awareness for seamless context transitions.

Multi-context window workflows

For tasks spanning multiple context windows:

1. **Use a different prompt for the very first context window:** Use the first context window to set up a framework (write tests, create setup scripts), then use future context windows to iterate on a todo-list.
2. **Have the model write tests in a structured format:** Ask Claude to create tests before starting work and keep track of them in a structured format (e.g., `tests.json`). This leads to better long-term ability to iterate. Remind Claude of the importance of tests: "It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality."
3. **Set up quality of life tools:** Encourage Claude to create setup scripts (e.g., `init.sh`) to gracefully start servers, run test suites, and linters. This prevents repeated work when continuing from a fresh context window.
4. **Starting fresh vs compacting:** When a context window is cleared, consider starting with a brand new context window rather than using compaction. Claude's latest models are extremely effective at discovering state from the local filesystem. In some cases, you may want to take advantage of this over compaction. Be prescriptive about how it should start:
    - "Call pwd; you can only read and write files in this directory."
    - "Review progress.txt, tests.json, and the git logs."
    - "Manually run through a fundamental integration test before moving on to implementing new features."
5. **Provide verification tools:** As the length of autonomous tasks grows, Claude needs to verify correctness without continuous human feedback. Tools like Playwright MCP server or computer use capabilities for testing UIs are helpful.

6. **Encourage complete usage of context:** Prompt Claude to efficiently complete components before moving on:

Sample prompt

```
This is a very long task, so it may be beneficial to plan out your work clearl
```

State management best practices

- **Use structured formats for state data:** When tracking structured information (like test results or task status), use JSON or other structured formats to help Claude understand schema requirements
- **Use unstructured text for progress notes:** Freeform progress notes work well for tracking general progress and context
- **Use git for state tracking:** Git provides a log of what's been done and checkpoints that can be restored. Claude's latest models perform especially well in using git to track state across multiple sessions.
- **Emphasize incremental progress:** Explicitly ask Claude to keep track of its progress and focus on incremental work

> **Example: State tracking**

Balancing autonomy and safety

Without guidance, Claude Opus 4.6 may take actions that are difficult to reverse or affect shared systems, such as deleting files, force-pushing, or posting to external services. If you want Claude Opus 4.6 to confirm before taking potentially risky actions, add guidance to your prompt:

Sample prompt

```
Consider the reversibility and potential impact of your actions. You are encou

Examples of actions that warrant confirmation:
- Destructive operations: deleting files or branches, dropping database tables
- Hard to reverse operations: git push --force, git reset --hard, amending pub
- Operations visible to others: pushing code, commenting on PRs/issues, sendin

When encountering obstacles, do not use destructive actions as a shortcut. For
```

Research and information gathering

Claude's latest models demonstrate exceptional agentic search capabilities and can find and synthesize information from multiple sources effectively. For optimal research results:

1. **Provide clear success criteria:** Define what constitutes a successful answer to your research question
2. **Encourage source verification:** Ask Claude to verify information across multiple sources
3. **For complex research tasks, use a structured approach:**

Sample prompt for complex research

```
Search for this information in a structured way. As you gather data, develop s
```

This structured approach allows Claude to find and synthesize virtually any piece of information and iteratively critique its findings, no matter the size of the corpus.

Subagent orchestration

Claude's latest models demonstrate significantly improved native subagent orchestration capabilities. These models can recognize when tasks would benefit from delegating work to specialized subagents and do so proactively without requiring explicit instruction.

To take advantage of this behavior:

1. **Ensure well-defined subagent tools:** Have subagent tools available and described in tool definitions
2. **Let Claude orchestrate naturally:** Claude will delegate appropriately without explicit instruction
3. **Watch for overuse:** Claude Opus 4.6 has a strong predilection for subagents and may spawn them in situations where a simpler, direct approach would suffice. For example, the model may spawn subagents for code exploration when a direct grep call is faster and sufficient.

If you're seeing excessive subagent use, add explicit guidance about when subagents are and aren't warranted:

Sample prompt for subagent usage

```
Use subagents when tasks can run in parallel, require isolated context, or inv
```

Chain complex prompts

With adaptive thinking and subagent orchestration, Claude handles most multi-step reasoning internally. Explicit prompt chaining (breaking a task into sequential API calls) is still useful when you

need to inspect intermediate outputs or enforce a specific pipeline structure.

The most common chaining pattern is **self-correction:** generate a draft → have Claude review it against criteria → have Claude refine based on the review. Each step is a separate API call so you can log, evaluate, or branch at any point.

Reduce file creation in agentic coding

Claude's latest models may sometimes create new files for testing and iteration purposes, particularly when working with code. This approach allows Claude to use files, especially python scripts, as a 'temporary scratchpad' before saving its final output. Using temporary files can improve outcomes particularly for agentic coding use cases.

If you'd prefer to minimize net new file creation, you can instruct Claude to clean up after itself:

Sample prompt

```
If you create any temporary new files, scripts, or helper files for iteration,
```

Overeagerness

Claude Opus 4.5 and Claude Opus 4.6 have a tendency to overengineer by creating extra files, adding unnecessary abstractions, or building in flexibility that wasn't requested. If you're seeing this undesired behavior, add specific guidance to keep solutions minimal.

For example:

Sample prompt to minimize overengineering

```
Avoid over-engineering. Only make changes that are directly requested or clear

- Scope: Don't add features, refactor code, or make "improvements" beyond what

- Documentation: Don't add docstrings, comments, or type annotations to code y

- Defensive coding: Don't add error handling, fallbacks, or validation for sce

- Abstractions: Don't create helpers, utilities, or abstractions for one-time
```

Avoid focusing on passing tests and hard-coding

Claude can sometimes focus too heavily on making tests pass at the expense of more general solutions, or may use workarounds like helper scripts for complex refactoring instead of using standard tools directly. To prevent this behavior and ensure robust, generalizable solutions:

Sample prompt

```
Please write a high-quality, general-purpose solution using the standard tools

Focus on understanding the problem requirements and implementing the correct a

If the task is unreasonable or infeasible, or if any of the tests are incorrec
```

Minimizing hallucinations in agentic coding

Claude's latest models are less prone to hallucinations and give more accurate, grounded, intelligent answers based on the code. To encourage this behavior even more and minimize hallucinations:

Sample prompt

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references a speci
</investigate_before_answering>
```

Capability-specific tips

Improved vision capabilities

Claude Opus 4.5 and Claude Opus 4.6 have improved vision capabilities compared to previous Claude models. They perform better on image processing and data extraction tasks, particularly when there are multiple images present in context. These improvements carry over to computer use, where the models can more reliably interpret screenshots and UI elements. You can also use these models to analyze videos by breaking them up into frames.

One technique that has proven effective to further boost performance is to give Claude a crop tool or skill. Testing has shown consistent uplift on image evaluations when Claude is able to "zoom" in on relevant regions of an image. Anthropic has created a cookbook for the crop tool.

Frontend design

Claude Opus 4.5 and Claude Opus 4.6 excel at building complex, real-world web applications with strong frontend design. However, without guidance, models can default to generic patterns that create what users call the "AI slop" aesthetic. To create distinctive, creative frontends that surprise and delight:

For a detailed guide on improving frontend design, see the blog post on improving frontend design through skills.

Here's a system prompt snippet you can use to encourage better frontend design:

Sample prompt for frontend aesthetics

```
<frontend_aesthetics>
You tend to converge toward generic, "on distribution" outputs. In frontend de

Focus on:
- Typography: Choose fonts that are beautiful, unique, and interesting. Avoid
- Color & Theme: Commit to a cohesive aesthetic. Use CSS variables for consist
- Motion: Use animations for effects and micro-interactions. Prioritize CSS-on
- Backgrounds: Create atmosphere and depth rather than defaulting to solid col

Avoid generic AI-generated aesthetics:
- Overused font families (Inter, Roboto, Arial, system fonts)
- Clichéd color schemes (particularly purple gradients on white backgrounds)
- Predictable layouts and component patterns
- Cookie-cutter design that lacks context-specific character

Interpret creatively and make unexpected choices that feel genuinely designed
</frontend_aesthetics>
```

You can also refer to the full skill definition.

Migration considerations

When migrating to Claude 4.6 models from earlier generations:

1. **Be specific about desired behavior:** Consider describing exactly what you'd like to see in the output.
2. **Frame your instructions with modifiers:** Adding modifiers that encourage Claude to increase the quality and detail of its output can help better shape Claude's performance. For example, instead of "Create an analytics dashboard", use "Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation."
3. **Request specific features explicitly:** Animations and interactive elements should be requested explicitly when desired.
4. **Update thinking configuration:** Claude 4.6 models use adaptive thinking (`thinking: {type: "adaptive"}`) instead of manual thinking with `budget_tokens` . Use the effort parameter to control thinking depth.
5. **Migrate away from prefilled responses:** Prefilled responses on the last assistant turn are no

longer supported starting with Claude 4.6 models. See Migrating away from prefilled responses for detailed guidance on alternatives.

6. **Tune anti-laziness prompting:** If your prompts previously encouraged the model to be more thorough or use tools more aggressively, dial back that guidance. Claude 4.6 models are significantly more proactive and may overtrigger on instructions that were needed for previous models.

For detailed migration steps, see the Migration guide.

Migrating from Claude Sonnet 4.5 to Claude Sonnet 4.6

Claude Sonnet 4.6 defaults to an effort level of `high`, in contrast to Claude Sonnet 4.5 which had no effort parameter. Consider adjusting the effort parameter as you migrate from Claude Sonnet 4.5 to Claude Sonnet 4.6. If not explicitly set, you may experience higher latency with the default effort level.

Recommended effort settings:

- **Medium** for most applications
- **Low** for high-volume or latency-sensitive workloads
- Set a large max output token budget (64k tokens recommended) at medium or high effort to give the model room to think and act

When to use Opus 4.8 instead: For the hardest, longest-horizon problems (large-scale code migrations, deep research, extended autonomous work), Opus 4.8 remains the right choice. Sonnet 4.6 is optimized for workloads where fast turnaround and cost efficiency matter most.

If you're not using extended thinking

If you're not using extended thinking on Claude Sonnet 4.5, you can continue without it on Claude Sonnet 4.6. You should explicitly set effort to the level appropriate for your use case. At `low` effort with thinking disabled, you can expect similar or better performance relative to Claude Sonnet 4.5 with no extended thinking.

Python

```python
client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=8192,
    thinking={"type": "disabled"},
    output_config={"effort": "low"},
    messages=[{"role": "user", "content": "..."}],
)
```

If you're using extended thinking

If you're using extended thinking with `budget_tokens` on Claude Sonnet 4.5, it is still functional on Claude Sonnet 4.6 but is deprecated. Migrate to adaptive thinking with the effort parameter.

Migrating to adaptive thinking

Adaptive thinking is particularly well suited to the following workload patterns:

- **Autonomous multi-step agents:** coding agents that turn requirements into working software, data analysis pipelines, and bug finding where the model runs independently across many steps. Adaptive thinking lets the model calibrate its reasoning per step, staying on path over longer trajectories. For these workloads, start at `high` effort. If latency or token usage is a concern, scale down to `medium`.
- **Computer use agents:** Claude Sonnet 4.6 achieved best-in-class accuracy on computer use evaluations using adaptive mode.
- **Bimodal workloads:** a mix of easy and hard tasks where adaptive skips thinking on simple queries and reasons deeply on complex ones.

When using adaptive thinking, evaluate `medium` and `high` effort on your tasks. The right level depends on your workload's tradeoff between quality, latency, and token usage.

Python

```
client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=64000,
    thinking={"type": "adaptive"},
    output_config={"effort": "high"},
    messages=[{"role": "user", "content": "..."}],
)
```

Keeping budget_tokens during migration

If you need to keep `budget_tokens` temporarily while migrating, a budget around 16k tokens provides headroom for harder problems without risk of runaway token usage. This configuration is deprecated and will be removed in a future model release.

For coding use cases (agentic coding, tool-heavy workflows, code generation), start with `medium` effort:

Python

```
client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16384,
    thinking={"type": "enabled", "budget_tokens": 16384},
    output_config={"effort": "medium"},
    messages=[{"role": "user", "content": "..."}],
)
```

For chat and non-coding use cases (chat, content generation, search, classification), start with `low`

effort:

```python
client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=8192,
    thinking={"type": "enabled", "budget_tokens": 16384},
    output_config={"effort": "low"},
    messages=[{"role": "user", "content": "..."}],
)
```

Was this page helpful?

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy